

NebulaVLA: A Dual-Frequency Vision-Language-Action Model With Guide Action for Robotic Manipulation

Cong Zhao, Shuai Tian, Xu Zhang, Baocheng Ni, Xinguo Song, Xueying Sun, Shu Jiang, Shouchang Yang
Bo Tang, Jin Deng, Ge Zhu, YongCheng Wang*, Jin Xu, Ri Yang

ZTE Corporation

China

wang.yongcheng@zte.com.cn

*Corresponding author

Abstract: Real-world deployment of Vision-Language-Action (VLA) models is often bottlenecked by efficiency-performance trade-offs, cross-embodiment generalization, and execution smoothness. We present NebulaVLA, an asynchronous dual-frequency architecture that decouples high-level semantic reasoning from low-level action control, optimizing computational resources and modularity. To bridge semantic gaps across heterogeneous robots, we introduce GESTURE-7, a unified language-grounded action representation. Furthermore, our Guide Action algorithm enforces kinematic continuity via mask-based smoothness constraints. Comprehensive evaluations demonstrate that NebulaVLA significantly outperforms synchronous baselines, achieving an 85.5% average success rate on LIBERO-Plus and accelerating action generation by $\sim 2.7\times$. This asynchronous design enables highly efficient and responsive control for practical robotics.

Keywords: Vision-Language-Action Model, Asynchronous Dual-Frequency Architecture, Guide Action

1 Introduction

Robotic manipulation is transitioning from modular pipelines to end-to-end learning, led by Vision-Language-Action (VLA) models. By unifying perception, language, and control within a single architecture, VLAs enable natural language instruction following and complex task execution with enhanced generalization [1, 2]. The field has advanced rapidly: since RT-1 [3] established the Transformer-based policy foundation, OpenVLA [4] accelerated community progress via scalable open-source development, and the InternVLA series [5, 6] introduced precise spatial grounding and a unified understanding-generation-action paradigm.

Contemporary VLA models, however, face three bottlenecks. First, monolithic synchronous architectures enforce a rigid efficiency-performance trade-off: uniform temporal resolution cannot concurrently support low-frequency semantic grounding and high-frequency motor control. Second, inadequate embodiment representation impedes cross-morphology generalization; mapping continuous kinematics to discrete tokens demands expansive vocabularies and large-scale demonstration data to span the action space, hindering unified action semantics across heterogeneous platforms. Third, maintaining action smoothness and continuity during execution remains challenging. The temporal mismatch between asynchronous real-time inference and action chunk generation often induces pronounced jitter and execution discontinuities.

We introduce **NebulaVLA**, a dual-frequency VLA model grounded in a fast-slow cognitive paradigm [7], designed to systematically address these limitations.

1. **Heterogeneous Dual-Frequency Architecture.** Deliberative reasoning (System 2, 10 Hz VLM) is decoupled from reactive control (System 1, 20 Hz DiT generator), optimizing the compute-latency and enabling distributed deployment.
2. **Unified Representation & Three-Stage Training.** We propose **GESTURE-7**, a 7D continuous end-effector vector that unifies action semantics via natural language keyword mapping which enables RL. Training follows a three-stage pipeline: Stage I and II sequentially optimize System 2 via supervised fine-tuning (SFT) and GRPO-based [8] RL to establish robust semantic planning; Stage III performs System 1 SFT using a weighted flow-matching loss to distill priors of S2, ensuring strict training-inference frequency alignment.
3. **Guide Action Mechanism.** Inspired by *Image Outpainting* [9, 10], we condition next-chunk generation on executed trajectory anchors from the preceding chunk; mask constraints during diffusion denoising enforce semantic-guided boundary stitching, significantly improving trajectory continuity and smoothness.

Experiments demonstrate that NebulaVLA significantly reduces inference latency while boosting task success rates over synchronous baselines. **GESTURE-7** markedly enhances cross-embodiment transferability, and Guide Action effectively suppresses real-world execution jitter (quantitative results in Sec. 4).

2 Related Work

VLA Architectures and Frequency Design. VLA models have evolved from foundational Transformer architectures [3] to large-scale frameworks integrating spatial grounding and unified perception-action paradigms, including knowledge-transfer variants [1, 2], open-source scalable designs [4], and unified understanding-generation-action systems such as $\pi 0$ series [11, 12, 13], InternVLA series [5, 6] and GR series [14, 15, 16]. Despite this progress, these approaches struggle to reconcile the computational overhead of semantic reasoning with the low-latency demands of high-frequency motor control. Recent efforts have begun exploring fast-slow system decoupling: academically, HiRT [17] conditions high-frequency policies on asynchronously cached VLM features; industrially, Figure Helix [18] adopts a similar dual-frequency hierarchy in real-world deployment. Both validate the efficacy of heterogeneous frequency design in balancing generalization with real-time execution. Aligned with this paradigm, NebulaVLA implements a structured dual-frequency architecture decoupled via explicit data-flow specifications. The design natively supports distributed deployment, minimizing interaction latency through asynchronous communication and workload separation, thereby establishing a scalable structural foundation for synchronizing high-level reasoning with low-level control.

Embodiment Representation and Action Tokenization. Cross-embodiment generalization fundamentally relies on unified action representation. Current discrete tokenization methods (FAST [19], Mind-to-Hand [20]) map continuous kinematics to categorical tokens via frequency bands or trajectory clustering, demanding extensive demonstrations and struggling with cross-morphology semantic alignment. Conversely, continuous approaches (PUMA/DOMINO [21]; KineVLA [22]) utilize optical flow or kinesthetic encoding but regress raw states without cross-platform semantic abstraction. NebulaVLA addresses this via **GESTURE-7**, a hybrid 7D end-effector representation that maps synonymous actions to shared natural language keywords. This formulation natively unifies heterogeneous embodiments (e.g., dexterous hands, grippers, arms) and seamlessly integrates with language-conditioned RL.

Trajectory Smoothness and Chunk Boundary Continuity. Ensuring motion continuity during asynchronous real-world deployment remains a critical bottleneck. While chunk-based generation (e.g., ACT [23]) simplifies policy learning, temporal misalignment between inference latency and control frequency induces severe jitter and boundary discontinuities. Existing solutions predominantly rely on *post-hoc* smoothing: spline-based interpolation [24, 25, 26], jerk-minimizing RL rewards [27], or detokenizer redesigns. Although effective in mitigating artifacts, these methods operate outside

the generative process and cannot guarantee semantic consistency across chunks. In contrast, NebulaVLA internalizes smoothness constraints via the **Guide Action** mechanism. Drawing inspiration from *Image Outpainting* [9, 10], we condition next-chunk generation on executed trajectory anchors and enforce mask constraints during diffusion denoising. This preserves known actions at each step, ensures semantic continuity at overlapping boundaries, and removes the need for post-processing smoothing.

3 Method

3.1 Architecture Overview

As illustrated in Fig.1, NebulaVLA employs a hierarchical fast-slow architecture with heterogeneous frequencies. System 2 performs high-level semantic planning by processing natural language instructions and sparse visual inputs (~ 10 Hz) to output abstract action guidance. System 1 operates as a low-level controller (~ 20 Hz), conditioning on this guidance with high-frequency visual inputs and action history. By explicitly decoupling these frequencies, our design balances computational efficiency with control responsiveness, natively supporting distributed deployment that minimizes latency in real-world interactions and maximizes hardware throughput.

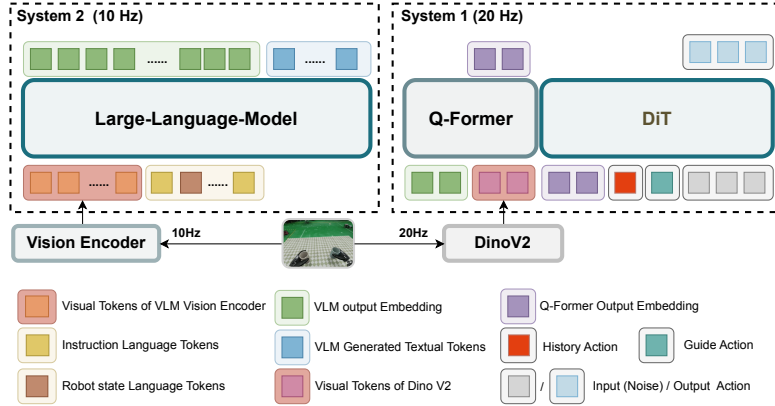


Figure 1: NebulaVLA Architecture. System 2 uses Qwen3-VL as backbone to output abstract action guidance from user instructions and sparse visual inputs. System 1 utilizes a Q-Former to compress S2’s guidance alongside high-frequency visual inputs, and ultimately outputs execution actions through a denoising process with a Diffusion Transformer.

3.2 NebulaVLA Components

System 2 (S2). S2 employs Qwen3-VL [28] as its vision-language backbone. It extracts hidden-layer features via learnable tokens to serve as abstract action guidance for S1.

System 1 (S1). Inspired by InternVLA-M1 [5], the design of S1 introduces two further optimizations: (1) contrastive fine-tuning of the pre-trained DINOv2 [29] using visual inputs and end-effector states to improve visual-proprioceptive alignment; and (2) integrating the Guide Action mechanism to ensure trajectory continuity and smoothness.

Guide Action. Asynchronous policy inference and action execution in real-world settings often compromise trajectory smoothness. To ensure seamless transitions between action chunks, we propose Guide Action. Inspired by *Image Outpainting* [9, 10], it embeds chunk concatenation directly into the generative pipeline, bypassing conventional RTC [30] and other post-hoc smoothing methods [23]. As shown in Fig.2a, a hard switch (red line) of trajectory for baseline methods causes mechanical jitter. Instead, we condition the next chunk’s generation on prior unexecuted actions

(e.g., a_4 – a_7 in Fig.2b). This enforces trajectory alignment (a'_4 – a'_7) for seamless boundary transitions. With guide action points number adapting to inference latency (randomly sampled from 0 to 3 during training), the denoising process is reformulated as the following conditional probabilistic model:

$$p_{\theta}(A_{full}^{t-1} \mid A_{full}^t, A_{prior}, \tau) \quad (1)$$

where:

- A_{full}^t is the complete action chunk at denoising timestep t (length T).
- A_{prior} denotes the prior action set, consisting of the executed history action (optional) and the guide action to be executed.
- τ represents the temporal positional query (positional encoding vector).

After each denoising step, similar to how *Image Outpainting* preserves original pixels, we forcibly reinstate the known actions:

$$A_{full}^{t-1} = M \odot A_{prior}^{(t-1)} + (1 - M) \odot A_{noisy}^{t-1} \quad (2)$$

where:

- M is a binary mask, $M \in \{0, 1\}^T$, such that:
 - $M[1..k] = 1$ (anchor region, strictly preserved),
 - $M[k + 1..T] = 0$ (generation region, predicted by the model);
- $A_{prior}^{(t-1)}$ is the noisy version of the known actions at diffusion timestep $t - 1$;
- A_{noisy}^{t-1} is the current denoised result predicted by the model.

and $\odot$ denotes element-wise multiplication.

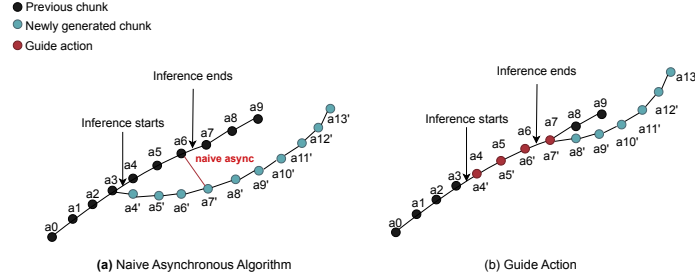


Figure 2: Scheme of the Guide Action mechanism. (a) The naive asynchronous baseline performs a hard switch (red line) after inference ends, leading to execution jitter. (b) Guide Action utilizes unexecuted prior steps (red dots) to condition the generation of the subsequent chunk, ensuring seamless trajectory alignment.

3.3 Training Process

NebulaVLA’s training pipeline comprises three stages: Stage 1 and Stage 2 perform supervised fine-tuning (SFT) and reinforcement learning (RL) on System 2 based on GESTURE-7 action description, respectively, while Stage 3 executes supervised fine-tuning on System 1.

GESTURE-7, short for Gesture Enumerated State Token for Unified Robot End-effector-7D, is a natural language-based method for describing end-effector states. Specifically, the current state of an end-effector is encoded as a 7-dimensional vector: $[x, y, z, r, p, w, gesture]$. In this vector, x, y, z represent the Cartesian position of the end-effector, while r, p, w correspond to its orientation (roll, pitch, yaw). The *gesture* component uses a natural language keyword to indicate the target hand posture. Different embodiments performing the same action share the same keyword—for example,

the grasping action, whether executed with a parallel gripper or a dexterous hand, is uniformly represented as *grasp*. Thus, GESTURE-7 unifies action descriptions across heterogeneous robots. More details are provided in Appendix A.

Stage 1: System 2 Supervised Fine-Tuning. Stage 1 establishes System 2’s visual-grounded proprioceptive understanding and trajectory prediction capabilities through supervised fine-tuning. The training mixture is organized into three task groups: (1) spatial grounding, including end-effector pose regression in GESTURE-7 format and 2D keypoint regression from visual observations; (2) trajectory forecasting, encompassing both 2D waypoint and 3D trajectory point prediction at 2 Hz; and (3) reasoning and planning, where the model generates chain-of-thought subtask decompositions paired with future GESTURE-7 trajectories, conditioned on visual inputs, language instructions, and current end-effector states. Details are provided in Appendix B.

Stage 2: System 2 Reinforcement Learning. Reinforcement learning has proven highly effective for aligning large language and vision-language models. Following Stage 1, we further enhance System 2’s trajectory planning capability through GRPO-based [8] reinforcement learning, retaining the same task suite as in supervised fine-tuning. The reward function for each task is detailed in Appendix C.

Stage 3: Supervised Fine-Tuning with Continuous Action. Stage 3 jointly optimizes the weights of both System 2 and System 1, while keeping their respective visual encoders frozen throughout training. We adopt an L2 loss to supervise the continuous action outputs. Owing to the guided action mechanism, the loss is restricted to the newly generated action region where the binary mask $M = 0$ (i.e., time steps $[k + 1..T]$ following the provided guide actions of length k):

$$\mathcal{L}(\theta) = E_{\epsilon, t} \left[\left\| (1 - M) \odot (\epsilon - \epsilon_{\theta}(x_t, t)) \right\|^2 \right] \quad (3)$$

Here, ϵ denotes the ground-truth noise, $\epsilon_{\theta}(\cdot)$ is the noise prediction from the model parameterized by θ , $M \in \{0, 1\}$ is a binary mask with $M = 1$ indicating the guide action context (positions $[1..k]$) and $M = 0$ indicating the newly generated region.

4 Experiment

4.1 Experiment Setup

Simulation Experiments. We employ LIBERO-Plus [31], a large-scale embodied manipulation benchmark designed to rigorously assess vision-language-action (VLA) policies under controlled environmental perturbations. Compared to the LIBERO [32] framework, LIBERO-Plus introduces 7 perturbation dimensions (encompassing 21 sub-dimensions), across 10,030 programmatically generated tasks categorized into five difficulty levels (L1–L5). We adopt the Success Rate as the primary performance metric. This includes success rates under individual and combined perturbation dimensions, as well as task-wise success rates.

Real-World Experiments. We conduct experiments to validate real-world deployment performance—particularly the effectiveness of the heterogeneous-frequency architecture and the Guide Action mechanism on AgiBot A2 robot platform, which features dual-arm manipulation capabilities suitable for complex industrial and domestic service scenarios. We select two representative manipulation tasks: Pick-and-Place and Packaging Line Material Feeding, corresponding to general grasping scenarios and structured industrial scenarios, respectively. Training is conducted on a single node with 8×H800 GPUs using a batch size of 96.

We employ Success Rate (SR) and Inference Latency to evaluate task completion capability and computational efficiency, and use Joint Jerk to measure trajectory smoothness. For each joint j , the mean absolute jerk is computed as:

$$\bar{J}_j = \frac{1}{N} \sum_{i=1}^N \left| \frac{q_{i+1,j} - 3q_{i,j} + 3q_{i-1,j} - q_{i-2,j}}{\Delta t^3} \right| \quad (4)$$

where $q_{i,j}$ denotes the angle of the j -th joint at the i -th action step along the trajectory. Δt is the control period, and N is the total number of trajectory points. The metric $\bar{J}_j$ is reported independently for each of the 7 joints in the right arm.

Baseline Methods. We compare NebulaVLA against state-of-the-art open-source VLA systems including InternVLA-M1 [5], $\pi 0.5$ [12], GR00T N1.5 [33], and OpenVLA-OFT [34]. We directly adopt the officially reported LIBERO-Plus results for those that have reported them. For methods lacking official results, we reproduce them using publicly released checkpoints and the training pipeline from LeRobot [35] library to ensure identical training configurations. All methods are evaluated under identical LIBERO-Plus observation spaces, action spaces, and evaluation protocols for a fair comparison.

4.2 Main Results

Simulation Results. As shown in Fig. 3, NebulaVLA achieves state-of-the-art robustness on LIBERO-Plus with an overall success rate of 85.5%, surpassing InternVLA-M1 (81.3%) by 4.2% and outperforming OpenVLA-OFT+ (79.6%), GR00T N1.5 (59.0%), and $\pi 0.5$ (58.0%).

NebulaVLA maintains near-optimal performance across dimensions: near-perfect Language scores, marked improvements over InternVLA-M1 and OpenVLA-OFT+ in Robot Initial State, $\sim 8\%$ gains in Layout, and comparable or superior results in Light, Background, and Noise. It further leads in Spatial, Object, and Goal (6% and 10% over InternVLA-M1 and OpenVLA-OFT+, respectively), matches InternVLA-M1 in Long while outperforming OpenVLA-OFT+ by $\sim 8\%$, and maintains 2–7% margins in Spatial and Object.

Notably, NebulaVLA exhibits marginally lower Camera performance than InternVLA-M1, attributed to GESTURE-7’s training hypothesis establishing fixed visual-spatial mappings (see ablation experiments).

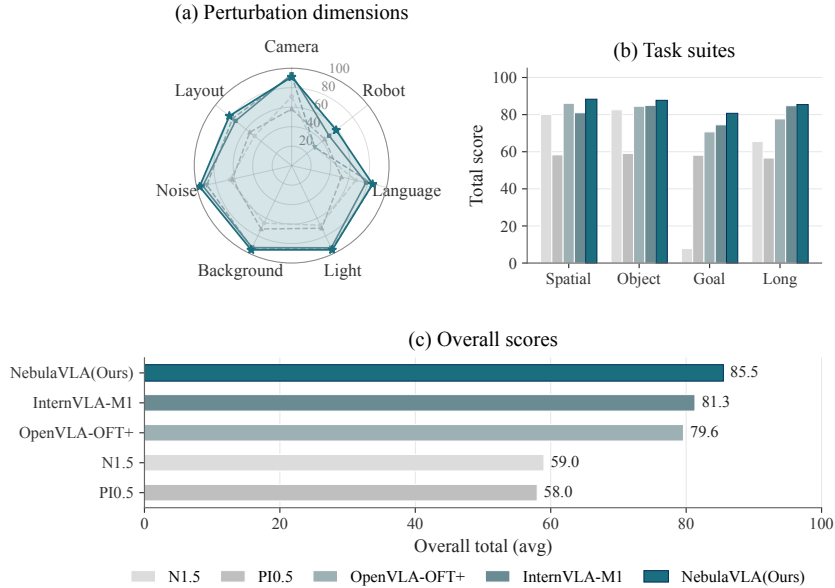


Figure 3: Simulation Result Comparison on Libero-Plus

Real-World Results. Real-world experiments are conducted on the AgiBot A2 robot platform with stereo RGB camera inputs. The policy is deployed on a NVIDIA RTX 4070 GPU. We select Task 1 (Pick-and-Place) and Task 2 (Packaging Line Material Feeding) for evaluation, comparing against InternVLA-M1. As shown in Table 1, NebulaVLA-Heterogeneous achieves the highest success rates in both tasks while maintaining the lowest inference latency.

	InternVLA-M1	NebulaVLA-Homo	NebulaVLA-Heter
Task 1: Pick-and-Place SR	77.91%	87.14%	92.08%
Task 2: Feeding SR	72.5%	90%	92.5%
Avg. Inference Latency (ms)	81	115	42

Table 1: Real-World Performance Comparison with InternVLA-M1

The heterogeneous-frequency structure significantly outperforms both the homogeneous-frequency variant and InternVLA-M1 in terms of success rate and latency. Compared to the homogeneous-frequency design, the heterogeneous architecture reduces the average step latency from 115 ms to 42 ms while improving success rates. This advantage stems from the decoupling of System 1 (fast action response) and System 2 (deep semantic reasoning): the high-frequency path focuses on lightweight action policy execution, while the low-frequency path handles scene understanding and task planning at longer intervals, thereby balancing efficiency and performance.

Furthermore, the heterogeneous-frequency architecture offers significant engineering advantages for real-world deployment. Since System 1 and System 2 operate independently, they can be deployed on separate computational devices. This enables NebulaVLA to flexibly adapt to edge computing scenarios, providing a practical solution for efficient VLA deployment in real robotic systems.

4.3 Ablation Studies

Training Strategy Ablation Study. To validate the effectiveness of our three-stage training strategy, we conduct ablation experiments across the seven perturbation dimensions (Camera, Robot, Language, Light, Background, Noise, Layout) on the LIBERO-Plus benchmark, with results presented in Table 2. NebulaVLA’s Stage 1 and Stage 2 perform S2 supervised fine-tuning (SFT) and reinforcement learning (RL), respectively, based on GESTURE-7, while Stage 3 executes S1 supervised fine-tuning. We configure three comparison settings: (1) NebulaVLA(w/o SFT, w/o RL): the S2 module directly loads open-source pretrained weights, omitting both GESTURE-7-based SFT and RL; (2) NebulaVLA(w/o RL): trained with GESTURE-7-based S2 SFT but omitting S2 RL; (3) NebulaVLA(ALL): complete three-stage trained.

Quantitative Analysis. NebulaVLA(ALL) achieves an overall success rate of 85.5%, representing 1.7% improvement over NebulaVLA(w/o SFT, w/o RL) (83.8%), and 2.5% improvement over NebulaVLA(w/o RL) (83.0%). This validates the effectiveness of progressive stage-wise optimization. Specifically, S2 SFT significantly improves performance in the Layout dimension (+6.2%), establishing reliable spatial priors for the policy; S2 RL further enhances performance in semantically-coupled spatial dimensions such as Language and Layout beyond the SFT-only baseline.

Interestingly, we observe performance degradation in the Camera perturbation dimension from both S2 SFT and RL. We attribute this to the influence of GESTURE-7’s training hypothesis, where its behavior representation implicitly establishes fixed mappings between visual features and spatial coordinates during training. Camera pose perturbations disrupt this correspondence, leading to degraded performance in this dimension. In future work, we plan to incorporate observation viewpoint conditioning into the GESTURE-7 training framework, explicitly modeling the mapping between viewpoint variations and spatial coordinates to further improve robustness under camera perturbations.

	Camera	Robot	Lang.	Light	Bkg.	Noise	Layout	Total
w/o SFT, w/o RL	93.5	57.9	80.5	97.8	94.9	92.6	73.0	83.8
w/o RL	87.8	56.3	82.8	93.3	93.2	94.2	79.2	83.0
NebulaVLA(ALL)	91.0	58.5	85.2	95.9	95.9	96.9	81.8	85.5

Table 2: Ablation Results of NebulaVLA on LIBERO-Plus

Guide Action Ablation Study As illustrated in Fig. 4, joint trajectory visualization reveals pronounced jumps at action chunk boundaries under the native asynchronous algorithm (left), where means actual outputs deviate from next predicted chunks (arrows), causing abrupt joint variations. The Guide Action algorithm (right) eliminates these discontinuities by introducing preceding guide points (red triangles) that align each new chunk’s starting state with the previous trajectory, ensuring smooth joint transitions.

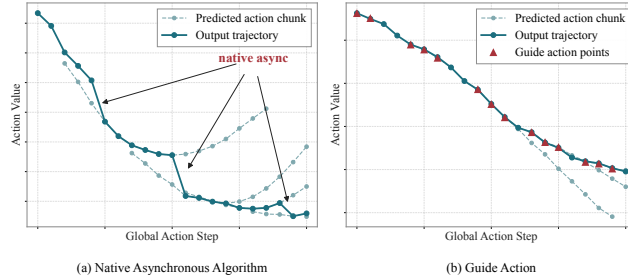


Figure 4: Visualization of *Guide Action* Effectiveness

Quantitative Analysis. Table 3 reports mean jerk ($\times 10^{-3}$) across joints 7–13 over 10 real-world trials. Guide Action reduces jerk in all joints, most substantially in joints 12 and 13 by 41.2% ($4.95 \rightarrow 2.91$) and 45.1% ($3.37 \rightarrow 1.85$), respectively, achieving an average reduction of 25.6% over the native asynchronous baseline. This validates that the Guide Action algorithm effectively suppresses boundary discontinuities and maintains kinematic continuity during chunk transitions.

Algorithm	Joint 7	Joint 8	Joint 9	Joint 10	Joint 11	Joint 12	Joint 13
Native Async	6.18	6.47	6.55	7.66	9.69	4.95	3.37
Guide Action	5.27	5.08	5.47	5.59	8.43	2.91	1.85

Table 3: Effect of *Guide Action* on Trajectory Smoothness

5 Conclusions and Limitations

Conclusions This paper presents **NebulaVLA**, a dual-frequency Vision-Language-Action model that systematically addresses three core challenges in robotic manipulation: computational efficiency versus control fidelity, cross-embodiment action representation, and trajectory continuity in real-world deployment. Our key contributions are: (1) a heterogeneous dual-frequency architecture that decouples semantic planning (~ 10 Hz) from reactive motor control (~ 20 Hz), enabling modular deployment and explicit compute-latency optimization; (2) **GESTURE-7**, a 7D continuous end-effector representation that unifies action semantics across heterogeneous platforms via natural language keyword mapping and natively supports reinforcement learning; and (3) the **Guide Action** mechanism, which enforces smoothness constraints intrinsically during diffusion denoising to suppress boundary jitter without post-processing.

Limitations Experimental results demonstrate that NebulaVLA consistently outperforms synchronous baselines across diverse manipulation tasks in both simulation and real-world deployment, validating the efficacy of heterogeneous frequency coordination. Nevertheless, several limitations warrant further investigation. First, the current frequency allocation is empirically determined and static; optimal configurations may vary across task domains, and dynamic adaptive scheduling remains an open challenge. Second, while Guide Action effectively suppresses inter-chunk discontinuities, its reliance on fixed guide anchor points introduces minor response latency in rapidly changing environments. Third, practical deployment still faces engineering hurdles in heterogeneous hardware synchronization, multi-sensor timestamp alignment, and distributed clock consistency.

Acknowledgments

This work was supported by the National Key Research and Development Program of China under Grant No. 2025YFB4712700, for the project "Research and Application Demonstration of Trustworthy Lightweight Large Models for Dual-Arm Mobile Robots."

References

- [1] B. Zitkovich, T. Yu, S. Xu, P. Xu, T. Xiao, F. Xia, J. Wu, P. Wohlhart, S. Welker, A. Wahid, et al. Rt-2: Vision-language-action models transfer web knowledge to robotic control. In *Conference on Robot Learning*, pages 2165–2183. PMLR, 2023.
- [2] D. Driess, F. Xia, M. S. Sajjadi, C. Lynch, A. Chowdhery, B. Ichter, A. Wahid, J. Tompson, Q. Vuong, T. Yu, et al. Palm-e: An embodied multimodal language model. *arXiv preprint arXiv:2303.03378*, 2023.
- [3] A. Brohan, N. Brown, J. Carbajal, Y. Chebotar, J. Dabis, C. Finn, K. Gopalakrishnan, K. Hausman, A. Herzog, J. Hsu, et al. Rt-1: Robotics transformer for real-world control at scale. *arXiv preprint arXiv:2212.06817*, 2022.
- [4] M. J. Kim, K. Pertsch, S. Karamcheti, T. Xiao, A. Balakrishna, S. Nair, R. Rafailov, E. Foster, G. Lam, P. Sanketi, et al. Openvla: An open-source vision-language-action model. *arXiv preprint arXiv:2406.09246*, 2024.
- [5] X. Chen, Y. Chen, Y. Fu, N. Gao, J. Jia, W. Jin, H. Li, Y. Mu, J. Pang, Y. Qiao, et al. Internvla-m1: A spatially guided vision-language-action framework for generalist robot policy. *arXiv preprint arXiv:2510.13778*, 2025.
- [6] J. Cai, Z. Cai, J. Cao, Y. Chen, Z. He, L. Jiang, H. Li, H. Li, and other. Internvla-a1: Unifying understanding, generation and action for robotic manipulation, 2026. URL <https://arxiv.org/abs/2601.02456>.
- [7] D. Kahneman. *Thinking, fast and slow*. macmillan, 2011.
- [8] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. Li, Y. Wu, et al. Deepseekmath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [9] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer. High-resolution image synthesis with latent diffusion models, 2021.
- [10] A. Lugmayr, M. Danelljan, A. Romero, F. Yu, R. Timofte, and L. Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11461–11471, 2022.
- [11] K. Black, N. Brown, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, L. Groom, K. Hausman, B. Ichter, et al. π_0 : A vision-language-action flow model for general robot control. *arXiv preprint arXiv:2410.24164*, 2024.
- [12] P. Intelligence, K. Black, N. Brown, J. Darpinian, K. Dhabalia, D. Driess, A. Esmail, M. Equi, C. Finn, N. Fusai, et al. $\pi_0.5$: a vision-language-action model with open-world generalization. *arXiv preprint arXiv:2504.16054*, 2025.
- [13] P. Intelligence, B. Ai, A. Amin, R. Aniceto, A. Balakrishna, G. Balke, K. Black, G. Bokinsky, S. Cao, T. Charbonnier, et al. $\pi_0.7$: a steerable generalist robotic foundation model with emergent capabilities. *arXiv preprint arXiv:2604.15483*, 2026.
- [14] H. Wu, Y. Jing, C. Cheang, G. Chen, J. Xu, X. Li, M. Liu, H. Li, and T. Kong. Unleashing large-scale video generative pre-training for visual robot manipulation. In *International Conference on Learning Representations*, volume 2024, pages 10641–10662, 2024.

- [15] C.-L. Cheang, G. Chen, Y. Jing, T. Kong, H. Li, Y. Li, Y. Liu, H. Wu, J. Xu, Y. Yang, et al. Gr-2: A generative video-language-action model with web-scale knowledge for robot manipulation. *arXiv preprint arXiv:2410.06158*, 2024.
- [16] C. Cheang, S. Chen, Z. Cui, Y. Hu, L. Huang, T. Kong, H. Li, Y. Li, Y. Liu, X. Ma, et al. Gr-3 technical report. *arXiv preprint arXiv:2507.15493*, 2025.
- [17] J. Zhang, Y. Guo, X. Chen, Y.-J. Wang, Y. Hu, C. Shi, and J. Chen. Hirt: Enhancing robotic control with hierarchical robot transformers. *arXiv preprint arXiv:2410.05273*, 2024.
- [18] Figure AI. Helix: A vision-language-action model for generalist humanoid control. Online, 2026. URL <https://www.figure.ai/news/helix>.
- [19] K. Pertsch, K. Stachowicz, B. Ichter, D. Driess, S. Nair, Q. Vuong, O. Mees, C. Finn, and S. Levine. Fast: Efficient action tokenization for vision-language-action models. *arXiv preprint arXiv:2501.09747*, 2025.
- [20] P. Tang, S. Xie, B. Sun, B. Huang, K. Luo, H. Yang, W. Jin, and J. Wang. Mind to hand: Purposeful robotic control via embodied reasoning. *arXiv preprint arXiv:2512.08580*, 2025.
- [21] H. Fang, S. Li, S. Wang, X. Xi, D. Liang, and X. Bai. Towards generalizable robotic manipulation in dynamic environments. *arXiv preprint arXiv:2603.15620*, 2026.
- [22] G. Han, Z. Gao, Z. Li, J. Huang, S. Huang, F. Karay, M. Gong, and T. Liu. Kinevla: Towards kinematics-aware vision-language-action models with bi-level action decomposition. *arXiv preprint arXiv:2603.17524*, 2026.
- [23] T. Z. Zhao, V. Kumar, S. Levine, and C. Finn. Learning fine-grained bimanual manipulation with low-cost hardware. *arXiv preprint arXiv:2304.13705*, 2023.
- [24] F. Yang, P. Jing, K. Qu, N. Zhao, and Y. Su. Abpolicy: Asynchronous b-spline flow policy for real-time and smooth robotic manipulation. *arXiv preprint arXiv:2602.23901*, 2026.
- [25] Z. Huang, M. Liu, X. Lin, M. Zhu, C. Zhao, Z. Du, Y. Lin, X. Li, Y. Jia, H. Zhong, et al. Notvla: Semantics-preserving robot adaptation via narrative action interfaces. *arXiv preprint arXiv:2510.03895*, 2025.
- [26] Y. Zhao, L. Zhao, B. Cheng, G. Yao, X. Wen, and H. Gao. Vla-rail: A real-time asynchronous inference linker for vla models and robots. *arXiv preprint arXiv:2512.24673*, 2025.
- [27] J. Li, X. Shi, H. Xie, M. Shang, and Y. Lu. Smoothvla: Aligning vision-language-action models with physical constraints via intrinsic smoothness optimization. *arXiv preprint arXiv:2603.13925*, 2026.
- [28] S. Bai, Y. Cai, R. Chen, K. Chen, X. Chen, Z. Cheng, L. Deng, W. Ding, C. Gao, C. Ge, et al. Qwen3-vl technical report. *arXiv preprint arXiv:2511.21631*, 2025.
- [29] M. Oquab, T. Darcet, T. Moutakanni, H. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, et al. Dinov2: Learning robust visual features without supervision. *arXiv preprint arXiv:2304.07193*, 2023.
- [30] K. Black, M. Galliker, and S. Levine. Real-time execution of action chunking flow policies. *Advances in Neural Information Processing Systems*, 38:33383–33407, 2026.
- [31] S. Fei, S. Wang, J. Shi, Z. Dai, J. Cai, P. Qian, L. Ji, X. He, S. Zhang, Z. Fei, J. Fu, J. Gong, and X. Qiu. Libero-plus: In-depth robustness analysis of vision-language-action models. *arXiv preprint arXiv:2510.13626*, 2025.
- [32] B. Liu, Y. Zhu, C. Gao, Y. Feng, Q. Liu, Y. Zhu, and P. Stone. Libero: Benchmarking knowledge transfer for lifelong robot learning. *arXiv preprint arXiv:2306.03310*, 2023.

- [33] J. Bjorck, F. Castañeda, N. Cherniadev, X. Da, R. Ding, L. Fan, Y. Fang, D. Fox, F. Hu, S. Huang, et al. Gr00t n1: An open foundation model for generalist humanoid robots. *arXiv preprint arXiv:2503.14734*, 2025.
- [34] M. J. Kim, C. Finn, and P. Liang. Fine-tuning vision-language-action models: Optimizing speed and success. *arXiv preprint arXiv:2502.19645*, 2025.
- [35] R. Cadene, S. Alibert, A. Soare, Q. Gallouedec, A. Zoutine, S. Palma, P. Kooijmans, M. Aractingi, M. Shukor, D. Aubakirova, M. Russi, F. Capuano, C. Pascal, J. Choghari, J. Moss, and T. Wolf. Lerobot: State-of-the-art machine learning for real-world robotics in pytorch. <https://github.com/huggingface/lerobot>, 2024.
- [36] F. Zhang, V. Bazarevsky, A. Vakunov, A. Tkachenka, G. Sung, C.-L. Chang, and M. Grundmann. Mediapipe hands: On-device real-time hand tracking. *arXiv preprint arXiv:2006.10214*, 2020.
- [37] P. Senin. Dynamic time warping algorithm review. *Information and Computer Science Department University of Hawaii at Manoa Honolulu, USA*, 855(1-23):40, 2008.

A GESTURE-7: Natural Language-Based End-Effector State Representation

We propose **GESTURE-7** (Gesture Enumerated State Token for Unified Robot End-effector-7D), a natural language-based representation for describing end-effector states. Formally, the state of an end-effector is encoded as a 7-dimensional vector:

$$\mathbf{s} = [x, y, z, r, p, w, g]$$

where $(x, y, z) \in \mathbb{Z}^3$ denote the Cartesian position in centimeter units, $(r, p, w) \in \mathbb{Z}^3$ represent orientation angles (roll, pitch, yaw) in integer degrees, and $g \in \mathcal{G}$ is a discrete gesture token from a predefined vocabulary $\mathcal{G}$ (e.g., "grasp", "push", "rotate", "release"). The gesture component uses natural language keywords to indicate the target hand posture, enabling different robot embodiments performing the same action to share identical keywords. This unified representation facilitates cross-embodiment action generalization.

Implementation details. End-effectors exhibit morphological variations; we select representative keypoints for pose representation: the middle finger root for dexterous hands and the gripper center for parallel grippers. To account for mechanical tolerances and hardware imprecisions, we intentionally quantize position predictions to integer centimeters and orientation angles to the nearest degree. This design choice reduces the learning burden on the model and mitigates prediction hallucination.

Extensibility. The gesture vocabulary $\mathcal{G}$ is not fixed; it can be freely extended based on task requirements. **Figure 5** illustrates common gesture keywords for both dexterous hands (subfigures 1–7) and grippers (subfigures 8–9).

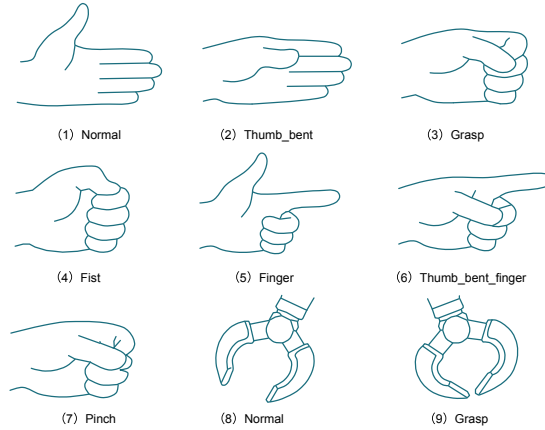


Figure 5: Illustration of common gesture keywords in GESTURE-7 end-effector states. Subfigures (1–7) depict gestures for dexterous hand end-effectors, while subfigures (8–9) depict those for gripper end-effectors.

B System 2 Supervised Fine-Tuning

Unlike prior works that incorporate massive visual grounding data or extensive cross-embodiment robot trajectories during pre-training, we observe that large-scale cross-embodiment manipulation data and multimodal perception capabilities yield limited transfer to VLA model training for our target tasks. Moreover, such extensive pre-training may lead to catastrophic forgetting or performance degradation of the original multimodal foundation model. To preserve the model’s general capabilities while enabling effective task-specific adaptation, we directly fine-tune on task-specific

data using the original Qwen3-VL [28] weights as initialization, bypassing additional pre-training stages.

Specifically, we devise five supervised fine-tuning tasks across three categories, all aligned with System 2’s two core capabilities: (1) **natural language-based state perception**, and (2) **GESTURE-7-based trajectory representation and prediction**. The task formulations are detailed below:

1. **Pose Regression:** Given visual observations, predict the current end-effector state in GESTURE-7 format.
2. **Keypoint Regression:** From visual observations, regress 2D hand keypoint coordinates [36].
3. **2D Waypoint Prediction:** From the current frame, predict future trajectory waypoints within the image plane at 2Hz.
4. **3D Trajectory Prediction:** Given the current visual frame, predict the robot’s end-effector GESTURE-7 trajectory at 2Hz over a future time horizon.
5. **Chain-of-Thought (CoT) Generation:** Given the current visual observations, instruction, and end-effector state, generate: (a) subtask reasoning process, (b) natural language subtask description, and (c) future GESTURE-7 trajectory.

These tasks collectively equip System 2 with both precise spatial perception and structured trajectory planning capabilities.

We illustrate the expected CoT output format in Listing 1.

Listing 1: Example of CoT generation output for Task 5.

Question: <Multi-view observation> Task: Stack second-order magic cube on top of electrical tape by pinching with two fingers. Current left hand state: (33,21,14,120,74,95,'finger'). Current right hand state: (41,-28,19,5,82,17,'finger'). Output the subtask, reasoning process, and a 3D trajectory in GESTURE-7 format at 2 Hz.

Answer: Subtask: My right hand moves towards the magic cube while my left hand stays still. Reason: My right hand moves to grasp the magic cube for stacking, while my left hand stays still to stabilize the scene and avoid interference. This ensures precise pinching and safe placement atop the electrical tape, maintaining balance and task accuracy. Left_hand_trajectory: [(33,21,14,120,74,95,'finger'), (33,21,14,120,74,95,'finger'), (33,21,14,120,74,95,'finger'), (33,21,14,120,74,95,'finger')]. Right_hand_trajectory: [(40,-27,23,-64,67,-46,'finger'), (40,-25,24,-87,45,-61,'thumb_bent_finger'), (41,-23,19,-104,28,-64,'thumb_bent_finger'), (42,-21,17,-115,16,-61,'thumb_bent_finger')].

C Reward Functions of System 2 Reinforcement Learning

Reinforcement learning has demonstrated significant effectiveness in training large language models and vision-language multimodal models. In this work, after System 2 Supervised Fine-Tuning, we employ GRPO [8] to enhance its trajectory planning capability. During this stage, we further reinforce the five tasks from Stage 1 with specialized reward functions. The reward functions are defined as follows:

GESTURE-7 Distance. Two end-effector states as GESTURE-7 vectors take the form:

$$\mathbf{s}_i = [x_i, y_i, z_i, r_i, p_i, w_i, g_i]^\top$$

$$\mathbf{s}_j = [x_j, y_j, z_j, r_j, p_j, w_j, g_j]^\top$$

where $(x, y, z) \in \mathbb{Z}^3$ denote the quantized Cartesian position (cm), $(r, p, w) \in \mathbb{Z}^3$ represent orientation angles (roll, pitch, yaw) in integer degrees, and $g \in \mathcal{G}$ is a discrete gesture token from a predefined enumeration set $\mathcal{G}$ (e.g., {grasp, push, rotate, release}).

The distance between two GESTURE-7 states, $D(\mathbf{s}_i, \mathbf{s}_j)$, is defined as the weighted sum of three components: **position distance**, **orientation distance**, and **gesture consistency penalty**.

$$D(\mathbf{s}_i, \mathbf{s}_j) = \lambda_p \cdot d_p(\mathbf{s}_i, \mathbf{s}_j) + \lambda_o \cdot d_o(\mathbf{s}_i, \mathbf{s}_j) + \lambda_g \cdot d_g(\mathbf{s}_i, \mathbf{s}_j)$$

where $\lambda_p, \lambda_o, \lambda_g \geq 0$ are weighting coefficients (set to 1.0 each in this work for balanced contributions). Specifically, the position distance $d_p(\mathbf{s}_i, \mathbf{s}_j)$ is defined as the Euclidean distance in Cartesian space; the orientation distance $d_o(\mathbf{s}_i, \mathbf{s}_j)$ is defined as the Euclidean distance in 3D angular space. The gesture distance penalizes mismatches between discrete gesture keywords:

$$d_g(\mathbf{s}_i, \mathbf{s}_j) = \begin{cases} 0, & \text{if } g_i = g_j \\ 1, & \text{if } g_i \neq g_j \end{cases}$$

This is a binary (0/1) gesture mismatch indicator.

Keypoint Localization Reward. For the keypoint detection task, the reward function R_{loc} measures the accuracy of predicted keypoints relative to ground-truth annotations:

$$R_{\text{loc}} = \frac{1}{K} \sum_{i=1}^K \exp \left(-\frac{\|\mathbf{k}_i^{\text{pred}} - \hat{\mathbf{k}}_i\|_2^2}{\sigma^2} \right)$$

where K is the number of keypoints [36], $\mathbf{k}_i^{\text{pred}} \in \mathbb{R}^2$ denotes the predicted 2D coordinates of the i -th keypoint, $\hat{\mathbf{k}}_i \in \mathbb{R}^2$ is the corresponding ground-truth annotation, and $\sigma > 0$ is a bandwidth parameter that controls the smoothness of the exponential penalty (larger σ yields more forgiving rewards).

Waypoints Reward. Following the approach of Mind to Hand [20], the waypoints reward combines point-level Euclidean distance with trajectory-level Dynamic Time Warping (DTW) distance [37] to evaluate trajectory quality.

Trajectory Reward. End-effector actions are described using the GESTURE-7 format, and the 3D trajectory-level reward is constructed using point-wise GESTURE-7 distance:

$$R_{\text{traj}} = \frac{1}{K} \sum_{i=1}^K (1 - D(\mathbf{s}_i, \hat{\mathbf{s}}_i))$$

where $\hat{\mathbf{s}}_i$ denotes the ground-truth GESTURE-7 state at trajectory point i , and $\mathbf{s}_i$ represents the model’s predicted state. Here, i indexes actions sampled at 2Hz. In practice, K typically takes values from $\{2, 3, 4\}$ (corresponding to a temporal horizon of 1–2 seconds). This design reflects our observation that future action predictions become inherently more diverse and uncertain at longer time horizons, making distant trajectory points prone to hallucination.

Format Reward. To ensure structural consistency between inference mode and subtask stages, regex-based matching is employed to enforce a predefined output format. A binary reward (1 or 0) is assigned based on whether the model’s output conforms to the predefined format template. This format reward contributes to the total reward with a weight of 0.5.